NoSQL Database Design – Lab Sheet 5

1\. Riak KV – Student Bucket

```bash

# Install and start Riak on Linux using the package

sudo systemctl start riak

# Check status

riak-admin cluster status

```

```bash

# Store student record

curl -X PUT http://localhost:8098/buckets/students/keys/101 \

  -H "Content-Type: application/json" \

  -d '{"name":"Amit","course":"BCA","marks":85}'

# Retrieve

curl http://localhost:8098/buckets/students/keys/101

# Update

curl -X PUT http://localhost:8098/buckets/students/keys/101 \

  -H "Content-Type: application/json" \

  -d '{"name":"Amit","course":"BCA","marks":90}'

# Delete

curl -X DELETE http://localhost:8098/buckets/students/keys/101

```

2\. Riak KV – User Sessions

```bash

curl -X PUT http://localhost:8098/buckets/user\_sessions/keys/S101 \
```

```
  -H "Content-Type: application/json" \
  -d '{"session_id":"S101","user_id":101,"login_time":"2026-10-29 10:00","status":"active"}'
curl http://localhost:8098/buckets/user\_sessions/keys/S101
```

3\. Riak KV – User Profiles and Preferences

```bash
curl -X PUT http://localhost:8098/buckets/profiles/keys/U101 \
  -H "Content-Type: application/json" \
  -d '{"name":"Amit","email":"amit@example.com","city":"Delhi","language":"Hindi","theme":"dark","notifications":true}'
curl http://localhost:8098/buckets/profiles/keys/U101
```

4\. Riak KV – Shopping Cart

```bash
# Add products to a user's cart
curl -X PUT http://localhost:8098/buckets/carts/keys/U101 \
  -H "Content-Type: application/json" \
  -d '{"items":[{"product_id":1,"quantity":2,"price":500},{"product_id":2,"quantity":1,"price":1000}],"total":2000}'
# Retrieve cart
curl http://localhost:8098/buckets/carts/keys/U101
# Update quantity and total
curl -X PUT http://localhost:8098/buckets/carts/keys/U101 \
  -H "Content-Type: application/json" \
```

```
  -d '{"items":[{"product_id":1,"quantity":3,"price":500},
{"product_id":2,"quantity":1,"price":1000}],"total":2500}'

# Remove product by storing the revised cart

curl -X PUT http://localhost:8098/buckets/carts/keys/U101 \

  -H "Content-Type: application/json" \

  -d '{"items":[{"product_id":2,"quantity":1,"price":1000}],"total":1000}'
```

5\. Riak KV – Key-Value Data Structure

```bash
# String value

curl -X PUT http://localhost:8098/buckets/demo/keys/name \

  -H "Content-Type: text/plain" -d 'Amit'

# JSON document

curl -X PUT http://localhost:8098/buckets/demo/keys/student \

  -H "Content-Type: application/json" \

  -d '{"id":101,"name":"Amit"}'

# List value stored as JSON

curl -X PUT http://localhost:8098/buckets/demo/keys/courses \

  -H "Content-Type: application/json" \

  -d '["BCA","Python","Java"]'

# Retrieve values

curl http://localhost:8098/buckets/demo/keys/name

curl http://localhost:8098/buckets/demo/keys/student

curl http://localhost:8098/buckets/demo/keys/courses
```

Explanation: A bucket groups records, a key uniquely identifies a record within its bucket, and a value contains the stored data.

6\. Riak KV – Consistency and Quorum

```bash

# Write with write quorum

curl -X PUT "http://localhost:8098/buckets/students/keys/101?w=2&dw=2" \

  -H "Content-Type: application/json" \

  -d '{"name":"Amit","marks":85}'

# Read with read quorum

curl "http://localhost:8098/buckets/students/keys/101?r=2"

```

Explanation: `w` specifies the number of replicas required to acknowledge a write, `dw` the number required to confirm durable storage, and `r` the number of replicas required for a read. These settings must be supported by the cluster's replication configuration.

7\. Riak KV – Horizontal Scaling

```bash

# Insert many records

for i in $(seq 1 1000)

do

  curl -s -X PUT "http://localhost:8098/buckets/users/keys/U$i" \

    -H "Content-Type: application/json" \

    -d "{\"id\":”$User$ i\"}" > /dev/null

done

# Insert into another bucket
```

```
curl -X PUT http://localhost:8098/buckets/products/keys/P101 \
  -H "Content-Type: application/json" \
  -d '{"name":"Laptop","price":50000}'
```

Explanation: Riak distributes data across nodes using consistent hashing. Additional nodes can increase storage and processing capacity, subject to cluster configuration.

8\. Riak KV – Transaction-Like Shopping Operations

```bash
# Initial cart
curl -X PUT http://localhost:8098/buckets/carts/keys/U101 \
  -H "Content-Type: application/json" \
  -d '{"product":"Mouse","quantity":1,"price":500,"total":500}'

# Update quantity and total
curl -X PUT http://localhost:8098/buckets/carts/keys/U101 \
  -H "Content-Type: application/json" \
  -d '{"product":"Mouse","quantity":3,"price":500,"total":1500}'

# Retrieve updated cart
curl http://localhost:8098/buckets/carts/keys/U101
```

Explanation: These are separate read/write operations, not a multi-step atomic transaction. Concurrent updates may conflict, so applications must handle conflicts and consistency explicitly.

9\. Riak KV – Query by User Data

```bash
# Store user records
curl -X PUT http://localhost:8098/buckets/users/keys/U101 \
  -H "Content-Type: application/json" \
```

```
  -d '{"name":"Amit","city":"Delhi","age":20,"preference":"dark"}'

curl -X PUT http://localhost:8098/buckets/users/keys/U102 \
  -H "Content-Type: application/json" \
  -d '{"name":"Riya","city":"Mumbai","age":22,"preference":"light"}'
```

```python
import requests

base = "http://localhost:8098/buckets/users/keys/"

keys = ["U101", "U102"]

for key in keys:
    user = requests.get(base + key).json()
    if user.get("city") == "Delhi":
        print(user)
```

Explanation: This example retrieves known keys and filters records in application code. It does not search all Riak records automatically; scalable attribute-based queries need an appropriate index or search feature.

10\. Riak KV – Set Operations

```bash
# Add elements to a Riak Set CRDT

curl -X POST \
  http://localhost:8098/types/sets/buckets/interests/datatypes/U101 \
  -H "Content-Type: application/json" \
  -d '{"add":["music","travel","gaming"]}'

# Retrieve the set

curl http://localhost:8098/types/sets/buckets/interests/datatypes/U101
```

```
# Remove an element

curl -X POST \

 http://localhost:8098/types/sets/buckets/interests/datatypes/U101 \

 -H "Content-Type: application/json" \

 -d '{"remove":["gaming"]}'
```

Explanation: Riak Set CRDTs support adding and removing elements. Membership can be checked against the retrieved set, and common elements between two sets can be calculated in application code. This requires a compatible Riak version with data types enabled.

11\. Neo4j – Student, Course and Department Graph

```cypher
CREATE (s1:Student {name:'Amit', id:101}),

    (s2:Student {name:'Riya', id:102}),

    (c:Course {name:'BCA'}),

    (d:Department {name:'Computer Science'});

MATCH (s:Student {id:101}), (c:Course {name:'BCA'})

CREATE (s)-[:ENROLLED_IN]->(c);

MATCH (c:Course {name:'BCA'}), (d:Department)

CREATE (c)-[:BELONGS_TO]->(d);

MATCH (s:Student)-[r]->(n)

RETURN s, r, n;
```

Explanation: Neo4j stores data as nodes and relationships. The final query returns the graph for visualization in Neo4j Browser.

12\. Neo4j – Social Network

```cypher
```

```
CREATE (a:Person {name:'Amit'}),
    (b:Person {name:'Riya'}),
    (c:Person {name:'Raj'}),
    (a)-[:FRIENDS_WITH]->(b),
    (c)-[:FOLLOWS]->(a),
    (a)-[:WORKS_AT]->(:Company {name:'ABC Ltd'});
```

```cypher
// Find Amit's friends
MATCH (:Person {name:'Amit'})-[:FRIENDS_WITH]-(f:Person)
RETURN f.name;
// Find Amit's followers
MATCH (f:Person)-[:FOLLOWS]->(:Person {name:'Amit'})
RETURN f.name;
// Find people Amit follows
MATCH (:Person {name:'Amit'})-[:FOLLOWS]->(p:Person)
RETURN p.name;
// Display the graph
MATCH (a:Person)-[r]->(b)
RETURN a, r, b;
```

13\. Neo4j – Shopping and E-Commerce Graph

```cypher
CREATE (c:Customer {name:'Amit'}),
    (p:Product {name:'Laptop', price:50000}),
```

```
    (cat:Category {name:'Electronics'}),
    (o:Order {orderId:101, total:50000});
MATCH (c:Customer {name:'Amit'}),
    (p:Product {name:'Laptop'}),
    (cat:Category {name:'Electronics'}),
    (o:Order {orderId:101})
CREATE (c)-[:PLACED]->(o),
    (o)-[:CONTAINS {quantity:1}]->(p),
    (p)-[:BELONGS_TO]->(cat);
```

```cypher
// Products purchased by Amit
MATCH (:Customer {name:'Amit'})-[:PLACED]->(:Order)-[:CONTAINS]->(p:Product)
RETURN p.name, p.price;
// Products by category
MATCH (p:Product)-[:BELONGS_TO]->(:Category {name:'Electronics'})
RETURN p.name;
```

14\. Neo4j – Transactions, Consistency and Availability

```cypher
// Execute as one transaction in Neo4j Browser
BEGIN;
CREATE (s:Student {id:101, name:'Amit'});
MATCH (s:Student {id:101})
SET s.marks = 90;
```

```
MATCH (s:Student {id:101})
CREATE (c:Course {name:'BCA'}),
    (s)-[:ENROLLED_IN]->(c);
COMMIT;
```

```cypher
// Delete a relationship
MATCH (:Student {id:101})-[r:ENROLLED_IN]->(:Course)
DELETE r;
// Verify data
MATCH (s:Student {id:101})
RETURN s;
```

Explanation:

* Transactions: Group changes together; commit applies them, while rollback cancels uncommitted changes.

* Consistency: Transactions and database constraints help maintain valid data.

* Availability: Depends on deployment, clustering, replication, and failover configuration.

15\. Neo4j – Graph Database Application Using Python

Install the Neo4j Python driver:

```bash
pip install neo4j
```

Python program (replace the password with your Neo4j password):

```python
```

```
from neo4j import GraphDatabase

driver = GraphDatabase.driver(
    "bolt://localhost:7687",
    auth=("neo4j", "your_password")
)

def create_student(tx):
    tx.run(
        "CREATE (:Student {id: $id, name: $name})",
        id=101, name="Amit"
    )

def read_students(tx):
    result = tx.run("MATCH (s:Student) RETURN s.name AS name")
    for record in result:
        print(record["name"])

def update_student(tx):
    tx.run(
        "MATCH (s:Student {id: $id}) SET s.name = $name",
        id=101, name="Amit Kumar"
    )

def delete_student(tx):
    tx.run("MATCH (s:Student {id: $id}) DETACH DELETE s", id=101)

with driver.session() as session:
    session.execute_write(create_student)
    session.execute_read(read_students)
    session.execute_write(update_student)
```

```
    session.execute_read(read_students)

    # Uncomment to delete the student:

    # session.execute_write(delete_student)

driver.close()
```

Explanation:

* CRUD: Create, read, update and delete nodes using Cypher.

* Relationship queries: Efficiently traverse connected nodes, such as friends or purchased products.

* Scaling and availability: Can be supported through appropriate clustering and replication configurations.

* Applications: Social networks, recommendation systems, fraud detection and knowledge graphs.